BiziSmart

Multi-Vertical SaaS Platform

Complete Technical & Product Documentation

Version 3.5 | October 2026 | Designed for Tanzanian SMEs

Pharmacy • Supermarket • Restaurant • Loans • Agro

5	PWA	M-Pesa	TRA	TSh 23K
Business verticals	Offline-first mobile app	Integrated payments	E-Fiscal ready (flagged)	Infrastructure cost/month (Phase 1, before WhatsApp)

Revision notes: v3.5 (React frontend restored on a Laravel API; v3.3 fixes kept)

#	Area	What changed	Where
1	Frontend	**React 18 + Redux Toolkit (RTK Query) + MUI + Vite + Workbox restored.** Livewire/Bootstrap removed from the client UI. Blade/Livewire remain only inside the Filament Super Admin.	2.2, 2.3, 19
2	Backend	Laravel is now a **JSON API** (Sanctum tokens, spatie permissions, Pennant flags, Horizon queues). **MySQL 8** kept. Node/Express and PostgreSQL are not returning.	2.2, 14, 19
3	Hosting layout	React build served as static files at `{business}.bizismart.co.tz`; the API lives at `/api/v1` on the same host (no CORS). Payment webhooks use `api.bizismart.co.tz` and find the tenant from the payment record.	14, 19.1, 19.5.1, 19.8
4	Offline	The React PWA is naturally offline-capable, but offline selling stays in Phase 3 after an online-first launch. Accept-and-flag rules unchanged.	12, 19.4
5	Kept	Laravel tenant scope + CI leak tests, honest cost model, shared-schema wording, TRA behind a feature flag, one vertical at a time.	10, 11, 12, 19.5

Table of Contents

Section	Title
1	Executive Summary
2	Platform Architecture
3	Super Admin Dashboard
4	Shared Core: Features Used by All Verticals
5	BiziSmart Pharmacy
6	BiziSmart Supermarket
7	BiziSmart Restaurant
8	BiziSmart Loans
9	BiziSmart Agro
10	Database Schema Overview
11	Hosting, Infrastructure and Costs
12	Development Roadmap
13	Conclusion
14	API Reference
15	UI/UX Wireframes
16	Error Handling and Edge Cases
17	Data Migration Strategy
18	Client Onboarding Flow
19	System Design

1 Executive Summary

BiziSmart is a cloud-based, offline-first SaaS platform built specifically for small and medium enterprises (SMEs) in Tanzania. It addresses operational gaps across five business verticals (Pharmacy, Supermarket, Restaurant, Loans/Microfinance, and Agro/Farm Inputs), replacing manual methods and inadequate legacy tools with an affordable, localized, mobile-first solution.

BiziSmart is structured as a monorepo: one shared Core that powers common functionality (POS, inventory, customer management, TRA compliance, WhatsApp/SMS, M-Pesa, offline sync), with five vertical-specific modules layered on top. Clients access their system via a custom subdomain (e.g. dawapharmacy.bizismart.co.tz). **Build order is one vertical at a time, starting with Pharmacy (Section 12).**

Core Value Proposition

Vertical	Serves	Headline features
Pharmacy	Duka la dawa, licensed pharmacies	FEFO, prescription logging, TFDA, NHIF claims
Supermarket	Supermarkets, mini-marts	Multi-till, departments, loss prevention, weighing scale
Restaurant	Restaurants, bars, food courts	Kitchen Display System, recipe costing, table management
Loans	Microfinance, Vicoba, Chama	Full loan lifecycle, PAR reports, field officer tools
Agro	Farm input shops, agro dealers	Farmer profiles, seasonal forecasting, harvest-linked credit

Technology Approach

BiziSmart is a **React PWA** (mobile-first, installable like an app) on top of a **Laravel API** with a MySQL database. One codebase serves phones, tablets and desktops. The phone camera is the barcode scanner, so no special hardware is needed. Offline selling (Phase 3) keeps the shop running through internet outages.

2 Platform Architecture

2.1 Monorepo Structure and Tenancy Model

All five verticals share a single codebase. The Core contains every feature common to all businesses. Each vertical is a module that extends the Core with industry-specific features.

- A bug fix in the Core is fixed for all five verticals simultaneously
- A new feature in Pharmacy does not affect Restaurant or Agro
- One server hosts all clients, dramatically reducing infrastructure cost
- You maintain one codebase, not five separate applications
- **Tenancy model: shared schema, row-level isolation.** All clients share the same tables. Every tenant table carries a `business_id`, and an Eloquent global scope (trait `BelongsToBusiness`) isolates each client's rows automatically and fails closed. CI tests make any leak a failed build (see 19.5.1).

Layer	Contents	Used by
Shared Core	Auth, POS engine, inventory, customer mgmt, supplier mgmt, employee mgmt, financial reports, TRA E-Fiscal (feature-flagged), WhatsApp/SMS, M-Pesa, offline sync, multi-branch, audit trail	All 5 verticals
Pharmacy Module	Prescription logging, FEFO enforcement, controlled substances register, TFDA, NHIF claims, dispensing labels, drug substitutes	Pharmacy only
Supermarket Module	Departments, multiple tills, loss prevention, weighing scale, promotions engine, customer display	Supermarket only
Restaurant Module	Table management, KDS, menu modifiers, recipe costing, ingredient depletion, reservations, bar tabs	Restaurant only
Loans Module	Borrower KYC, loan lifecycle, repayment schedules, PAR reports, field collection, group loans	Loans only
Agro Module	Farmer profiles, season management, harvest-linked credit, chemical register, field officer tools	Agro only
Super Admin Panel	Client management, feature toggles, role builder, subscription billing, platform analytics	Admin only

2.2 Technology Stack

Shape: a React single-page PWA (the face) talking to a Laravel JSON API (the brain). **All business rules live in Laravel**; React only presents data and collects input. That keeps the risky logic (stock, money, tenant isolation) in one place that is easy to test.

Layer	Technology	Why / role
Frontend framework	React 18 + Vite (JavaScript; TypeScript optional later)	Component UI, one codebase for all verticals and devices
UI components	MUI 5+	Professional, accessible, mobile-friendly components
State and data	Redux Toolkit + RTK Query	Cart and session state; API calls, caching and loading states in one library
Routing	React Router	Screens per role and vertical
PWA and offline (Phase 3)	Workbox (vite-plugin-pwa) + Dexie.js (IndexedDB)	Installable app; offline sales queue and local product lookup
Barcode scanning	`BarcodeDetector` API where supported, `@zxing/browser` fallback	iOS Safari lacks BarcodeDetector, so a fallback is required
Dashboards and charts	Recharts	Sales and report charts
Backend language and framework	PHP 8.3+ and Laravel (current stable), API-only	Routing, validation (Form Requests), API Resources, Eloquent ORM, queues, scheduler, testing

Layer	Technology	Why / role
API documentation	Scribe or Scramble (auto-generated from Laravel)	Keeps React and Laravel agreeing on request/response shapes
Database	MySQL 8 (InnoDB)	Familiar; transactions and row locks for stock accuracy
Multi-tenancy	Single database, `business_id` + Eloquent global scope (trait `BelongsToBusiness`)	Guarded by mandatory CI tests (19.5.1)
Authentication	Sanctum Bearer device tokens, Fortify TOTP 2FA (Owner and Super Admin), PIN login via custom guard	Tokens suit a SPA and offline use
Roles and permissions	spatie/laravel-permission (teams = business)	Role Builder; the API is the authority, the UI only hides actions
Feature toggles	Laravel Pennant (per business); React reads them from `GET /me/features`	Disabled features vanish from the UI and are blocked in the API
Super Admin panel	Filament at admin.bizismart.co.tz (server-rendered, same Laravel codebase)	Client, plan and feature management without building a second React app
Jobs and queues	Laravel queues on Redis + Horizon	Receipts, WhatsApp/SMS, TRA posting, reports
Scheduler	Laravel scheduler (one cron entry)	Nightly digests, M-Pesa reconciliation every 15 min, reminders
Cache, rate limits	Redis 7	Cache, queues, Laravel RateLimiter
File storage	Cloudflare R2 via Laravel S3 driver	Receipts, prescription photos, ID documents, backups
PDF and Excel	spatie/laravel-pdf or dompdf; maatwebsite/excel	Receipts, reports, exports
Audit trail	spatie/laravel-activitylog	Who did what, when, from which device
Payments	AzamPay / Selcom via Laravel HTTP client	M-Pesa, Tigo Pesa, Airtel Money
WhatsApp and SMS	Meta Business (Cloud) API and Beem Africa via Laravel notification channels	Owner alerts first; customer receipts opt-in (Section 11)
Backups	spatie/laravel-backup to R2, plus daily mysqldump	Encrypted off-server copies
Testing	Pest (API), Vitest + React Testing Library (components), Playwright (end-to-end)	Covers both halves
Web server	NGINX serves the React build and proxies `/api` to PHP-FPM; Supervisor runs Horizon	Standard Laravel deployment plus static hosting
Monitoring	UptimeRobot + Laravel Pulse (Sentry optional for React and Laravel errors)	Uptime alerts, slow queries, failed jobs, front-end errors
Dev workflow	One Git monorepo (`/backend` Laravel, `/frontend` React), GitHub Actions CI, local Herd/Laragon or GitHub Codespaces	Everything lives in Git so work is not tied to one machine

2.3 PWA and Mobile Strategy

Capability	How the PWA delivers it
Barcode scanning	Phone camera via `BarcodeDetector`, `@zxing/browser` fallback (needed on iPhone Safari)
Offline mode (Phase 3)	Workbox service worker + IndexedDB (Dexie): product and customer lookup and sales work without internet and sync on reconnect (19.4). Launch versions need internet
Install on home screen	Web app manifest; 'Add to Home Screen' looks and feels like a native app
Push notifications	Web Push for in-app alerts; owner alerts go primarily by WhatsApp
Bluetooth printing	Web Bluetooth (Chrome on Android/desktop). **Not available on iPhone Safari**: fall back to browser print or a shareable PDF receipt
Background sync	Service worker Background Sync where supported; a manual Sync button is always available
GPS capture	Geolocation API for farmer location and field officer visit stamps
Cross-platform	Android (Chrome), iPhone (Safari), tablets, laptops from one codebase

2.4 Trade-offs of This Stack

- **Gained:** a fast, installable app that feels native, a front end that can work offline, and a clean API that Enterprise clients and future mobile apps can reuse.
- **Cost:** two codebases to build, test and keep in agreement, and more to learn than server-rendered pages. Tenant safety depends on application code plus tests (MySQL has no row-level security), so those tests are mandatory.
- **How to keep it manageable:** design each endpoint's request and response first (Scribe/Scramble documents it), then build the Laravel side and the React screen against that contract. Keep every business rule in Laravel services. Build one thin vertical slice first (login, add product, sell, print receipt) before widening.
- **Upgrade paths:** PostgreSQL with RLS as a second isolation layer, Laravel Octane for speed, TypeScript in React once the API contract settles.

3 Super Admin Dashboard

The Super Admin Dashboard is your command centre. It lives at admin.bizismart.co.tz, completely separate from client-facing interfaces. No client can see or access it. All admin logins require mandatory 2FA. It is built with **Filament** inside the same Laravel codebase.

3.1 Admin Capabilities

Section	What you can do
Client Management	View all clients, onboard new clients, view per-client profile, usage, and activity
Feature Configuration	Toggle any feature ON/OFF per client (Laravel Pennant flags); disabled features disappear entirely from the client UI
Role Builder	Create and assign user roles per client with granular permission toggles
Subscription and Billing	Change plans, extend subscriptions, add free months, suspend accounts, record payments
Feature Templates	Create templates (e.g. 'Pharmacy Basic') and apply to new clients with one click
User Management	Search any user across all clients, view login history, force logout, reset credentials
Platform Analytics	Total clients, MRR, active users today, vertical breakdown, churn analysis (post-MVP)
System Health	Server status, database health, error logs per client, sync queue and sync conflict status

3.2 Admin Security

Control	Detail
Separate URL	admin.bizismart.co.tz; clients never see or know it exists
2FA mandatory	Two-factor authentication required for every admin login
Audit trail	Every action logged: who did what, when, from which IP
Session timeout	Auto-logout after 30 minutes of inactivity
IP whitelist (optional)	Restrict admin access to specific IP addresses only

4 Shared Core: Features Used by All Verticals

Module	Features
Authentication and Roles	PIN-based quick login, role-based dashboard views (spatie permissions), device management and approval, auto-logout, TOTP 2FA for owner
POS Engine	Phone camera barcode scanning, product search, multi-payment (cash/M-Pesa/split), receipt print or WhatsApp, offline POS, hold sale
Cash Till Management	Opening float entry, end-of-day cash count, expected vs actual reconciliation, variance flagging, supervisor approval
Inventory	Product catalogue, batch tracking, reorder alerts, stock take/cycle count, barcode label generation, purchase orders, GRN
Customer Management	Customer profiles, purchase history, credit limits, debt tracking, aging reports, loyalty points (post-MVP), QR loyalty card (post-MVP)
Supplier Management	Supplier database, purchase orders, goods received notes, payment scheduling, supplier price comparison (last 3 prices)
Employee Management	Role assignment, sales tracking, attendance logging; commissions and leaderboard are post-MVP
Financial Reports	P&L, cash flow, expense recording with photo, bank reconciliation, TRA-compliant tax reports, PDF and Excel export
TRA E-Fiscal	Electronic fiscal device integration, VRN/TIN support, tax report generation, filing reminders. Built behind a feature flag (Section 12)
WhatsApp and SMS	Owner alerts, daily morning digest, debt reminders, transaction receipts (customer WhatsApp is opt-in; see Section 11)
Offline-First Sync	Full functionality without internet, IndexedDB local storage, background sync on reconnect, accept-and-flag conflict handling (19.4). Offline selling is built in Phase 3
M-Pesa Integration	STK Push to customer, auto-matching of M-Pesa repayments, subscription fee collection via AzamPay/Selcom
Multi-Branch	Centralized dashboard, inter-branch stock transfer requests, consolidated reports (post-MVP)
AI Demand Forecasting	Moving average analysis, stockout prediction alerts, reorder suggestions (post-MVP)
Audit Trail	Every action logged with user, timestamp, device; searchable and exportable

5 BiziSmart Pharmacy

Designed for duka la dawa and licensed pharmacies. Enforces FEFO, captures prescriptions, manages NHIF claims, and maintains TFDA compliance. **This is the first vertical to be built.**

5.1 User Roles

Role	Access level
Owner / Proprietor	Full access: reports, financials, settings, all modules
Pharmacist	Dispense drugs, capture prescriptions, controlled substances, view stock; no financials
Cashier	POS only; no buying prices, no reports, no prescription drugs without pharmacist PIN
Stockkeeper	Receive stock, adjust stock, batch management; no sales reports

5.2 Pharmacy-Specific Features

Feature	Description
Prescription Logging	Patient name, doctor, facility, date, prescription number, photo (stored in Cloudflare R2)
FEFO Enforcement	First Expiry First Out: oldest batch always dispensed first; system enforces automatically
Controlled Substances Register	Schedule 1-3 separate log, printable for regulatory inspectors
NHIF Claims Management	Claim creation, batch submission report, status tracking, reconciliation
TFDA Registration Numbers	Each drug product stores its TFDA registration number
Drug Substitute Suggestions	When a drug is out of stock, system suggests equivalent alternatives
Drug Allergy Alerts	Patient allergy profile; red alert at POS if dispensing an allergen
Dispensing Label Printing	Patient name, dosage instructions, expiry; Bluetooth printer
Batch Recall Alerts	If a drug batch is recalled, all affected stock is flagged instantly and the owner gets a WhatsApp alert
Pharmacist on Duty Logging	Every dispensing transaction linked to the pharmacist on duty

5.3 Pricing Plans

Free	Basic	Professional	Enterprise
TSh 0	TSh 60,000/mo	TSh 120,000/mo	TSh 220,000/mo
1 user, 50 products, basic POS, BiziSmart watermark	2 users, full inventory, FEFO, expiry alerts, prescription logging	5 users, NHIF claims, controlled substances register, analytics	Unlimited users, multi-branch, TRA E-Fiscal, API access

6 BiziSmart Supermarket

Built for supermarkets and mini-marts. Multiple independent cashier tills, department management, loss prevention controls, and weighing scale integration.

6.1 Key Features

Feature	Description
Department Management	Departments (Dairy, Bakery, Beverages, etc.) with department-level stock and P&L reports
Multiple Cashier Tills	Each cashier has an independent till session with separate opening float and end-of-day reconciliation
Weighing Scale Integration	Bluetooth scale: weight auto-fills in cart; price = weight x price/kg
Loss Prevention Controls	Supervisor PIN for voids, refunds, large discounts; every override logged with reason and approver
Promotions Engine	Buy X Get Y, % discounts, fixed price promos, bulk discounts, time-based/weekend pricing
Customer Display Screen	Second screen facing customer showing cart total in real time
Suspicious Activity Alerts	If one cashier has too many voids in a day, the owner gets a WhatsApp alert automatically
Stock Take per Department	Stockkeeper scans shelves with phone, enters count; variances flagged in real time

6.2 Pricing Plans

Free	Basic	Professional	Enterprise
TSh 0	TSh 70,000/mo	TSh 150,000/mo	TSh 280,000/mo
1 till, 2 users, 200 products, basic POS	2 tills, 5 users, departments, full inventory	5 tills, 15 users, promotions, loss prevention, analytics	Unlimited tills/users, multi-branch, weighing scale, API

7 BiziSmart Restaurant

Front-of-house and back-of-house management: Kitchen Display System, table management, recipe costing, ingredient depletion, bar tabs, and delivery tracking.

7.1 Order Flow

Step	Action
1. Seat Customer	Walk-in: open table on floor plan. Reservation: system finds booking and assigns table automatically
2. Take Order	Waiter taps menu items on phone, adds modifiers (no onions, extra sauce), selects course, taps Send to Kitchen
3. Kitchen Receives	Order appears on the Kitchen Display System instantly with table number and timestamp. Kitchen marks Preparing, then Ready
4. Waiter Notified	Waiter gets a notification on phone when the order is ready
5. Additional Orders	Waiter opens the same table and adds items; only new items go to the kitchen; same running bill
6. Bill and Payment	Cashier reviews the bill, applies discounts (supervisor PIN), processes payment. Table marked Needs Cleaning

7.2 Restaurant-Specific Features

Feature	Description
Floor Plan Editor	Drag-and-drop table layout; sections (Indoor/Outdoor/Bar/VIP), capacity, coordinates
Kitchen Display System	Real-time orders on kitchen tablet; color-coded urgency timer; bump to dismiss when served
Recipe Costing	Each dish linked to ingredients with quantities; cost per dish calculated automatically
Ingredient Depletion	Every dish sold deducts exact ingredient quantities from stock
Dish Auto-Unavailable	When an ingredient runs out, every dish that uses it is marked unavailable on all waiters' phones
Split Bill	By person (equal), by item, or custom amounts
Bar Tab Management	Running tab per customer; close and pay at end
Reservation Calendar	Date, time, party size, customer name; walk-in queue alongside reservations

7.3 Pricing Plans

Free	Basic	Professional	Enterprise
TSh 0	TSh 80,000/mo	TSh 160,000/mo	TSh 300,000/mo
5 tables, 2 users, basic POS, no KDS	15 tables, 5 users, KDS, takeaway, bar tabs	Unlimited tables, 15 users, delivery, recipe costing, analytics	Multi-branch, API, reservations, advanced reports

8 BiziSmart Loans

Serves microfinance institutions, Vicoba groups, chama/table banking, salary advance businesses, and asset finance providers. Full loan lifecycle from KYC to closure.

8.1 Loan Lifecycle

Stage	Action
1. Borrower Registration	KYC: photo ID, selfie, national ID, phone, address, GPS location, occupation, guarantor details, digital signature
2. Loan Application	Loan officer selects borrower, purpose, amount, loan product; system calculates interest, repayment schedule, processing fee
3. Approval Workflow	Amount-based routing: small loans by branch manager; large loans by owner; committee loans need multiple approvers
4. Disbursement	Cash at office or M-Pesa direct to borrower; separate cashier authorization; repayment schedule activated
5. Repayment Collection	At office (cashier), in field (loan officer, offline), or M-Pesa self-service (auto-matched by phone number)
6. Overdue Management	Automatic WhatsApp/SMS escalation (Day 1, 3, 7); penalty calculation; guarantor notification; recovery officer assignment
7. Closure	Loan marked closed when fully repaid; collateral released; credit score updated; congratulations message sent

8.2 Key Loans Features

Feature	Description
Borrower KYC	Photo ID + selfie capture, national ID duplicate check, GPS location, blacklist check at registration
M-Pesa Auto-Matching	Payments matched by phone number and allocated automatically
Group Loans	Vicoba/chama group registration, group savings, group loan disbursement, meeting attendance log
PAR Reports	Portfolio At Risk: PAR30, PAR60, PAR90
Borrower Credit Scoring	Calculated from repayment history; higher score = higher credit limit on next loan
Field Collection	Loan officer collects cash offline in the field; receipts generated; data syncs on reconnect

8.3 Pricing Plans

Free	Basic	Professional	Enterprise
TSh 0	TSh 80,000/mo	TSh 180,000/mo	TSh 350,000/mo
10 active loans, 1 user, basic repayment tracking	100 active loans, 3 users, M-Pesa auto-matching, reminders	Unlimited loans, 10 users, PAR reports, group loans, field collection	Unlimited everything, multi-branch, investor reporting, API access

9 BiziSmart Agro

Serves farm input shops and agro dealers. Customers are known farmers with profiles. Demand is seasonal. Credit is tied to harvest cycles rather than immediate ability to pay.

9.1 The Agro Season Calendar

Season phase	System behaviour
6 weeks before planting peak	Alert owner to reorder seeds and fertilizer now; shows last season quantities as a guide
Planting season (peak)	Fast-moving stock tracked closely; daily reorder alerts; field officers see live stock before making promises
Growing season	Herbicides and pesticides peak; reorder focus shifts to these categories
Harvest season	System generates farmer debt collection list; field officers assigned per farmer; credit repayments expected
Off season	Slow-moving stock report; clearance pricing suggestions; plan next season orders

9.2 Agro-Specific Features

Feature	Description
Farmer Profiles	Name, national ID, phone, village, ward, GPS location, photo, crop types, acreage, irrigation type
Harvest-Linked Credit	Repayment date set to harvest date; debt flagged for collection when harvest season begins
Chemical Hazard Flags	Every pesticide/herbicide has hazard class (I-IV), storage requirements, safe handling instructions at POS
Chemical Sales Register	Regulatory requirement: every chemical sale logged with buyer, quantity, hazard class, handling confirmation
TPRI Registration Numbers	Tanzania Pesticides Research Institute registration number stored per chemical product
Field Officer Offline Tools	Farmer registration, order taking, repayment collection; works without internet in villages
GPS-Stamped Visits	Every field officer visit logged with GPS coordinates and timestamp
Season-Aware Reorder Alerts	'Planting season in 3 weeks: reorder these products now' based on last season sales velocity

9.3 Pricing Plans

Free	Basic	Professional	Enterprise
TSh 0	TSh 55,000/mo	TSh 130,000/mo	TSh 260,000/mo
1 user, 50 products, basic POS, no field officer tools	3 users, farmer profiles, credit sales, expiry tracking	10 users, field officers, offline sync, season management, chemical register	Unlimited users, multi-branch, advanced analytics, API access

10 Database Schema Overview

BiziSmart uses **MySQL 8 (InnoDB)**. Every tenant table has a `business_id` (bigint, indexed) column. Isolation is enforced by an Eloquent global scope that fails closed, plus CI tests (see 19.5.1). **Shared schema, row-level isolation.** Public-facing IDs (receipts, API) use ULIDs; internal keys are bigint.

10.1 Core Tables

Table	Purpose	Key fields
businesses	One row per client	id, public_id (ULID), name, vertical, plan, is_active, subdomain
branches	Physical locations per business	id, business_id, name, address, gps_lat, gps_lng
users	All users across platform	id, business_id, name, phone, pin_hash, is_active (roles via spatie/laravel-permission tables)
roles / permissions	spatie/laravel-permission tables (teams = business)	roles, permissions, model_has_roles, role_has_permissions (team_id = business_id)
products	Product/drug/ingredient catalogue	id, business_id, name, barcode, category, buying_price, selling_price
product_batches	Batch/expiry tracking per product	id, business_id, product_id, batch_number, expiry_date, qty_remaining (may go negative via sync; no CHECK constraint)
customers	Customer/patient/farmer/borrower base	id, business_id, name, phone, credit_limit, outstanding_balance
sales	Every transaction header	id, business_id, branch_id, cashier_id, total, payment_method, status (incl. committed_with_conflict), created_at
sale_items	Line items per sale	id, business_id, sale_id, product_id, batch_id, qty, unit_price, discount
till_sessions	Cashier till open/close records	id, business_id, branch_id, cashier_id, opening_float, closing_cash, variance
sync_operations	Idempotency log of device operations	operation_id (unique), business_id, device_id, type, created_at, processed_at, result
sync_conflicts (NEW)	Conflicts raised by accept-and-flag sync	id, business_id, operation_id, device_id, type (NEGATIVE_STOCK, PRICE_CHANGED, EXPIRED_BATCH, RECALLED_BATCH, QUARANTINED), product_id, batch_id, details JSON, status (open/resolved), resolved_by, created_at

10.2 Vertical-Specific Tables

Vertical	Additional tables
Pharmacy	prescriptions, prescription_items, controlled_substances_log, patients, nhif_claims, batch_recalls, pharmacist_duty_log
Supermarket	departments, tills, promotions, loss_prevention_log, stock_take_sessions, stock_take_items
Restaurant	restaurant_tables, floor_sections, reservations, menu_items, menu_modifiers, table_orders, order_items, kitchen_stations, comp_void_log, food_waste_log, bar_tabs
Loans	borrowers, guarantors, loan_products, loans, repayment_schedules, repayment_transactions, collateral, loan_groups, loan_writeoffs
Agro	farmers, farmer_crops, agro_seasons, season_products, chemical_sales_register, field_visits, demo_samples, farmer_credit_sales

11 Hosting, Infrastructure and Costs

BiziSmart launches on a single VPS hosting all verticals and all clients. Clients access their system via custom subdomains (e.g. dawapharmacy.bizismart.co.tz). As the platform grows, the server is upgraded or a second server is added, with no architectural changes required.

Before quoting any figure below: confirm current Hetzner plan names, specs and prices (the plan labelled CX21 may no longer be 4 vCPU / 8 GB), current Meta WhatsApp Business pricing for Tanzania, and current AzamPay/Selcom transaction fees.

11.1 Monthly Infrastructure Cost by Phase

Item	Provider	Phase 1 (0-20 clients)	Phase 2 (20-100)	Phase 3 (100-300)
VPS Server	Hetzner	TSh 15,600	TSh 31,200	TSh 62,400
Domain (.co.tz)	Truehost	TSh 2,500	TSh 2,500	TSh 2,500
SSL Certificate	Let's Encrypt	TSh 0	TSh 0	TSh 0
File Storage	Cloudflare R2	TSh 0	TSh 5,000	TSh 15,000
Email	Resend / Brevo	TSh 0	TSh 13,000	TSh 26,000
WhatsApp API	Meta Business API	**Budget required (not free)**	TSh 50,000	TSh 150,000
SMS	Beem Africa	TSh 5,000	TSh 20,000	TSh 50,000
TOTAL		**TSh 23,100 + WhatsApp**	**TSh 121,700**	**TSh 305,900**

11.2 Costs That Were Missing in v3.2

- **WhatsApp:** Meta charges for business-initiated template messages, so Phase 1 needs a real budget line. To keep it low, send the morning digest and alerts to owners only, make customer WhatsApp receipts opt-in, and default customers to printed receipts or SMS.
- **Payment gateway fees:** AzamPay and Selcom charge a percentage per transaction. Any subscription or customer payment collected through them reduces net revenue.
- **Your time and support:** onboarding, WhatsApp support and bug fixing are real costs, even if not cash.
- **Itemised startup cost** (replaces the single TSh 195,600 figure; list every item you actually pay for):

Startup item	Cost (TSh)	Notes
Domain (first year)		
First month VPS		
Meta Business verification / WhatsApp setup		
TRA / fiscal device requirements (if any)		Confirm with TRA
Test phone and Bluetooth printer		
Marketing / first client onboarding budget		
Total		Sum only what is listed

11.3 Revenue Model (replaces v3.2 projection)

Paid plans range from TSh 55,000 to TSh 350,000 and every vertical has a free tier, so revenue must be modelled with the share of clients who actually pay:

Item	Formula
Monthly revenue	clients x % paying x average paid plan
Gateway fees	revenue collected via gateway x fee %

Item	Formula
Infrastructure	VPS + domain + storage + email + SMS + WhatsApp (11.1)
Your time and support	hours per client per month x value of your hour
Monthly profit	revenue - gateway fees - infrastructure - your time

Illustrative example (assumes 60% of clients pay; replace with your real conversion data):

Phase	Clients	% paying	Avg paid plan	Revenue (illustrative)	Infrastructure
Phase 1	20	60%	TSh 80,000	TSh 960,000	TSh 23,100 + WhatsApp
Phase 2	100	60%	TSh 80,000	TSh 4,800,000	TSh 121,700
Phase 3	300	60%	TSh 85,000	TSh 15,300,000	TSh 305,900

Break-even statement (replaces 'profitable from month 3 with 2 clients'): infrastructure cost (TSh 23,100) is covered by a single paying client on the cheapest paid plan (TSh 55,000). Real profitability also depends on gateway fees, WhatsApp usage, support time, and the number of paying clients.

12 Development Roadmap

12.1 Build Order (one vertical at a time)

Principle: every phase ends with real users before the next begins. Launch **online-first** to get paying clients sooner; add offline selling once real users show where internet drops hurt most.

Phase	Build	Exit gate
1	Core, online-first. **First slice:** login, add product, sell, receipt, end to end. Then widen: Laravel API + tenant scope + leak tests, auth and roles, products and batches, POS, customers and debt, till management, basic reports, React app shell with MUI, Filament admin (clients, plans, feature flags)	Works end to end in a test shop
2	Pharmacy MVP: FEFO, prescription logging, expiry alerts, controlled substances register, M-Pesa STK push	3 to 5 paying pharmacies using it daily
3	Offline selling: Workbox + Dexie, accept-and-flag sync, Bluetooth printing	A pilot pharmacy keeps selling through an internet outage with no lost sale
4	Pharmacy hardening: NHIF claims, batch recall, WhatsApp owner alerts	Stable for 2 months, low support load
5	Second vertical (Loans or Agro, chosen by client demand)	3 to 5 paying clients
6+	Supermarket, Restaurant, Android Play Store	Only when earlier phases pay for themselves

12.2 MVP Scope

- **In MVP (Phases 1-2):** phone + PIN login, POS with camera scanning, inventory and batches, customers and debt, till management, basic reports, Filament admin (clients, subscriptions, feature flags).
- **Post-MVP:** offline selling, AI demand forecasting, multi-branch, loyalty and QR cards, employee commissions and leaderboard, feature templates, platform analytics.
- **TRA E-Fiscal:** confirm TRA's requirements and approval timeline now. Build it behind a Pennant feature flag so launch is never blocked, and switch it on per client later.

12.3 Testing Strategy

Layer	Tool	Target	Focus areas
API unit	Pest	80% on money-critical code	Sale totals, stock deduction, interest calculations, commissions, sync conflict rules
API feature (HTTP)	Pest + Laravel test client	Every endpoint	Responses, DB state, permissions, stock accuracy after sales
Tenant isolation	Pest	Mandatory, CI-blocking	Two businesses; every tenant model and route checked for leaks; every model with `business_id` must use the trait (19.5.1)
Frontend unit	Vitest + React Testing Library	70% minimum	Component logic, form validation, Redux state, cart maths
End-to-end	Playwright	10-15 critical paths	Login, cash sale, M-Pesa sale, stock adjustment, till close
Payment callbacks	Pest + HTTP fakes	100% webhook paths	Success, failure, timeout, duplicate callback (idempotency), tenant lookup from payment record
Offline sync	Pest + manual device tests	Critical scenarios	Offline sale, reconnect, two devices selling the last unit (both commit, conflict logged), price change, recalled batch

13 Conclusion

BiziSmart v3.5 is a multi-vertical SaaS platform that can serve five underserved business types in Tanzania from one codebase and one server, built one vertical at a time.

Success factor	How BiziSmart addresses it
Strong product-market fit	Solves real pain points for each specific vertical, not a generic solution
Infrastructure-aware design	Installable React PWA with offline selling (Phase 3) and accept-and-flag sync so no real sale is ever lost
Regulatory compliance	TRA E-Fiscal (flagged), TFDA, NHIF, chemical register: compliance becomes a reason customers stay
Communication-first UX	WhatsApp alerts and daily digest meet business owners where they already are (customer receipts opt-in to control cost)
Blue ocean verticals	Loans and Agro have very little dedicated software competition in Tanzania
Admin-controlled platform	You configure every feature for every client: clients get exactly what they need
Low infrastructure cost	TSh 23,100/month plus WhatsApp covers Phase 1 infrastructure; one paying client on the cheapest paid plan covers it
Realistic path to profit	Itemise startup costs, model paying share and fees (Section 11), and prove each vertical before building the next

Next step (Session 1): monorepo with `/backend` (Laravel) and `/frontend` (React + Vite + MUI); MySQL migrations for the core tables; the `BelongsToBusiness` trait, `ResolveTenant` middleware and tenant leak tests; then phone + PIN login end to end (API and login screen).

14 API Reference: REST Specification

All endpoints follow REST conventions. Base URL: `https://{business}.bizismart.co.tz/api/v1` (same host as the React app; tenant resolved from the subdomain). Payment webhooks use `https://api.bizismart.co.tz/v1`. Every request must include `Authorization: Bearer <device_token>`. All responses return JSON in the envelope defined in Section 16.1.

14.1 Authentication

The React app uses this API for everything. The business is identified by the **subdomain** (the React app and the API share one host, so requests carry no business ID and need no CORS). Tokens are Sanctum Bearer **device tokens** with an expiry and rotation. Web calls and offline syncs use the same endpoints.

Method	Endpoint	Description
POST	/auth/login	Phone + PIN on a registered device; returns a device token
POST	/auth/token/rotate	Rotate the device token before it expires
POST	/auth/logout	Revoke the current device token
POST	/auth/pin/reset	Owner resets a user's PIN
GET	/me	Current user, role, permissions
GET	/me/features	Enabled features (Pennant) for this business

POST /auth/login

```
// Request  (tenant resolved from the subdomain)
{ "phone": "0712345678", "pin": "1234", "device_id": "dev_xyz789" }

// Response
{ "token": "1|k3Jx...", "token_type": "Bearer", "expires_at": "2026-11-07T10:00:00Z",
  "user": { "id": "usr_001", "name": "Amina Hassan", "role": "cashier", "branch_id": "br_01" } }
```

14.2 POS and Sales

Method	Endpoint	Description
POST	/sales	Create a new sale (cash, M-Pesa, split)
GET	/sales	List sales; filter by date, cashier, branch
GET	/sales/{sale_id}	Full sale detail with line items
POST	/sales/{sale_id}/refund	Refund a sale (requires supervisor PIN)
POST	/sales/hold	Hold current sale, assign hold reference
GET	/sales/hold	List all held sales for this branch
DELETE	/sales/hold/{hold_id}	Discard a held sale

POST /sales

```
// Request
{ "branch_id": "br_01", "cashier_id": "usr_001", "customer_id": "cust_042",
  "payment_method": "split", "cash_amount": 5000, "mpesa_amount": 3500,
  "items": [ { "product_id": "prod_101", "batch_id": "bat_55",
               "qty": 2, "unit_price": 4000, "discount": 0 } ] }

// Response
{ "sale_id": "sale_9981", "receipt_number": "RCP-2026-9981", "total": 8500, "change": 0,
  "status": "completed", "receipt_url": "https://r2.../rcp9981.pdf",
  "whatsapp_sent": false, "created_at": "2026-04-12T10:34:00Z" }
```

14.3 Inventory

Method	Endpoint	Description
GET	/products	List products; search, filter by category
POST	/products	Create new product

Method	Endpoint	Description
PUT	/products/{product_id}	Update product details or price
DELETE	/products/{product_id}	Archive a product (soft delete)
GET	/products/{product_id}/batches	List all batches for a product
POST	/products/{product_id}/batches	Add new stock batch (GRN)
POST	/stock/take	Submit stock take count
GET	/stock/reorder-alerts	Products below reorder level
POST	/purchase-orders	Create purchase order to supplier
PUT	/purchase-orders/{po_id}/receive	Mark PO received; auto-adds batches

POST /products

```
// Request
{ "name": "Paracetamol 500mg", "barcode": "6123456789012", "category": "Analgesics",
  "buying_price": 800, "selling_price": 1200, "reorder_level": 50, "unit": "tablet",
  "tfda_reg": "TFDA-2021-0045" }          // tfda_reg: Pharmacy only

// Response
{ "product_id": "prod_887", "name": "Paracetamol 500mg", "barcode": "6123456789012",
  "current_stock": 0, "status": "active", "created_at": "2026-04-12T11:00:00Z" }
```

14.4 Customers

Method	Endpoint	Description
GET	/customers	List customers; search by name/phone
POST	/customers	Register new customer
GET	/customers/{customer_id}	Customer profile + purchase history
PUT	/customers/{customer_id}	Update customer details
GET	/customers/{customer_id}/debt	Outstanding debt and aging report
POST	/customers/{customer_id}/payment	Record manual debt payment

14.5 M-Pesa Integration

Method	Endpoint	Description
POST	/mpesa/stk-push	Trigger STK push to customer phone
POST	/mpesa/callback	AzamPay webhook on api.bizismart.co.tz; tenant found via checkout_request_id; auto-matches payment (idempotent)
GET	/mpesa/status/{checkout_id}	Poll STK push status
GET	/mpesa/transactions	List all M-Pesa transactions

```
// POST /mpesa/stk-push  Request
{ "phone": "255712345678", "amount": 8500, "sale_id": "sale_9981", "description": "BiziSmart Purchase" }
// Response
{ "checkout_request_id": "ws_CO_...", "merchant_request_id": "29115-...",
  "response_code": "0", "response_description": "Success", "customer_message": "Success. Request accepted." }
```

14.6 Pharmacy Endpoints

Method	Endpoint	Description
POST	/pharmacy/prescriptions	Log a new prescription
GET	/pharmacy/prescriptions	List prescriptions; filter by date/patient
GET	/pharmacy/controlled-substances	Controlled substances log (printable)
POST	/pharmacy/nhif/claims	Submit NHIF claim batch
GET	/pharmacy/nhif/claims	List NHIF claims and status
GET	/pharmacy/expiry-alerts	Products expiring within 30/60/90 days

Method	Endpoint	Description
POST	/pharmacy/batch-recall	Flag a drug batch as recalled

14.7 Restaurant Endpoints

Method	Endpoint	Description
GET	/restaurant/tables	Floor plan: all tables + status
POST	/restaurant/tables/{id}/order	Add items to table order
PUT	/restaurant/orders/{id}/status	Update order status (preparing/ready/served)
POST	/restaurant/tables/{id}/bill	Generate bill for table
POST	/restaurant/tables/{id}/split	Split bill by person / by item
POST	/restaurant/reservations	Create reservation
GET	/restaurant/kds	Kitchen Display System feed (live orders)
POST	/restaurant/bar-tabs	Open a bar tab
PUT	/restaurant/bar-tabs/{id}/close	Close and pay bar tab

14.8 Loans Endpoints

Method	Endpoint	Description
POST	/loans/borrowers	Register borrower with KYC
POST	/loans/applications	Submit loan application
PUT	/loans/{loan_id}/approve	Approve loan (role-based routing)
POST	/loans/{loan_id}/disburse	Disburse approved loan
POST	/loans/{loan_id}/repayment	Record repayment (cash or M-Pesa)
GET	/loans/par-report	PAR30/PAR60/PAR90 report
GET	/loans/overdue	Overdue loans with escalation status
POST	/loans/groups	Register Vicoba/chama group

14.9 Reports and Analytics

Method	Endpoint	Description
GET	/reports/sales	Sales summary by day/week/month
GET	/reports/profit-loss	P&L report; filter by date range
GET	/reports/cash-flow	Cash flow statement
GET	/reports/inventory-value	Current inventory valuation
GET	/reports/employee-performance	Sales per cashier, commissions
GET	/reports/export/{type}	Download PDF or Excel report

14.10 Offline Sync (NEW in v3.3)

Method	Endpoint	Description
POST	/sync/push	Device sends queued operations in order; each result is committed, committed_with_conflict, duplicate, quarantined or rejected
GET	/sync/pull?since=	Changes made by other devices or backend jobs since last sync
GET	/sync/conflicts	List open sync conflicts for the business (owner)
PUT	/sync/conflicts/{id}/resolve	Mark a conflict resolved with a note (e.g. after a stock count)

```
// POST /sync/push  Response (per operation)
{ "success": true,
  "data": { "results": [
```

```
{ "operation_id": "6f1e...", "status": "committed" },
{ "operation_id": "a02c...", "status": "committed_with_conflict",
  "conflict": { "type": "NEGATIVE_STOCK", "product_id": "prod_101", "qty_remaining": -2 } },
{ "operation_id": "9bd4...", "status": "duplicate" } ] } }
```

15 UI/UX Wireframes: Screen Layouts

Low-fidelity blueprints that define structure, not visual design. The actual UI uses Material-UI (MUI) components with BiziSmart's blue palette.

15.1 Login Screen

```
+-------------------------------------------------+
|                   BiziSmart                     |
|             Dawa Pharmacy - Mwanza              |
|  +-----------------------------------------+   |
|  |  Amina Hassan                           |   |
|  +-----------------------------------------+   |
|  |  John Mwakasege                         |   |
|  +-----------------------------------------+   |
|  |  Fatuma Omari                           |   |
|  +-----------------------------------------+   |
|              [Tap your name to login]           |
|   [1] [2] [3] [4] [5] [6] [7] [8] [9] [0] [OK]  |
+-------------------------------------------------+
```

15.2 POS Screen: Cashier View

```
+-------------------------------------------------------------+
| BiziSmart POS            [Online] [Amina] [Menu]            |
+--------------------------------------+----------------------+
| Search product or scan...            | CART                 |
| Paracetamol 500mg        TSh 1,200   | Paracetamol 500mg x2   |
| Amoxicillin 250mg        TSh 3,500   |                TSh 2,400|
| ORS Sachet               TSh   800   | ORS Sachet x1          |
| Metronidazole 400mg      TSh 2,000   |                TSh   800|
| Vitamin C 1000mg         TSh 1,500   |                       |
| Ibuprofen 400mg          TSh 1,800   | TOTAL:         TSh 3,200|
| [Scan Barcode]                       | [Cash] [M-Pesa]        |
|                                      | [Split] [Hold]         |
+--------------------------------------+----------------------+
```

15.3 Restaurant Floor Plan

```
+-------------------------------------------------------------+
| BiziSmart Restaurant           [+ New Table] [Reservations]   |
| [Indoor v] [Outdoor] [Bar] [VIP]                            |
+-------------------------------------------------------------+
|  +--------+  +--------+  +--------+  +--------+             |
|  | T1     |  | T2     |  | T3     |  | T4     |             |
|  | Free   |  | Active |  | Bill   |  | Free   |             |
|  | 4 pax  |  | 2 pax  |  | 6 pax  |  | 2 pax  |             |
|  +--------+  +--------+  +--------+  +--------+             |
|  +--------+  +--------+  +--------+  +--------+             |
|  | T5     |  | T6     |  | T7     |  | T8     |             |
|  | Active |  | Dirty  |  | Free   |  | Active |             |
|  +--------+  +--------+  +--------+  +--------+             |
|  Free | Active | Bill Ready | Needs Cleaning               |
+-------------------------------------------------------------+
```

15.4 Kitchen Display System

```
+-------------------------------------------------------------+
| KITCHEN DISPLAY                                  12:34 PM   |
+-----------------+------------------+--------------------+
| NEW        02:15 | PREPARING   05:41 | READY              |
| TABLE 2  #0042   | TABLE 5  #0041    | TABLE 3  #0039     |
| 2x Chips         | 1x Ugali          | 1x Beef Steak      |
| 1x Kuku          | 1x Samaki         | 2x Chips           |
| no pepper        | 2x Pilau          |                    |
| [START]          | [READY]           | [BUMP]             |
+-----------------+------------------+--------------------+
```

15.5 Loans Dashboard

```
+-------------------------------------------------------------+
| BiziSmart Loans                      [+ New Loan] [Profile]   |
+--------------+---------------+----------------+--------------+
| Active Loans | Disbursed     | Repaid Today   | Overdue      |
| 142          | TSh 45.2M     | TSh 1.8M       | 23           |
| +8 today     | this month    | 12 payments    | PAR30: 3%    |
+--------------+---------------+----------------+--------------+
| Search borrower by name or phone...                         |
| NAME            AMOUNT     DUE DATE     STATUS              |
| Juma Mwangi     TSh 500K   Apr 20       Due Soon            |
| Rehema Said     TSh 200K   Mar 15       Overdue             |
| Hassan Omari    TSh 1.2M   May 01       On Track            |
| [PAR Report] [Field Collection] [Group Loans] [Export]      |
+-------------------------------------------------------------+
```

15.6 Owner Dashboard

```
+-------------------------------------------------------------+
| BiziSmart Dawa Pharmacy                    [Alerts 3] [Menu]  |
| Good morning, Dr. Baraka!       Today: Sunday Apr 12, 2026    |
+--------------+--------------+-------------+-----------------+
| TODAY SALES   | GROSS PROFIT  | CUSTOMERS   | LOW STOCK       |
| TSh 284,000   | TSh 91,200    | 47          | 12 items        |
| +12% vs yest. | margin: 32%   |             |                 |
+--------------+--------------+-------------+-----------------+
| SALES THIS WEEK (bar chart Mon-Sun)                         |
| Open sync conflicts: 1  [Review]                            |
| [Full Reports] [Inventory] [Staff] [Settings]               |
+-------------------------------------------------------------+
```

15.7 Supermarket Multi-Till View (Manager)

```
+-------------------------------------------------------------+
| BiziSmart Supermarket [Manager View]           12:45 PM       |
| TILL 1 - Amina          TILL 2 - John                       |
| OPEN                    OPEN                                |
| Sales: TSh 124K         Sales: TSh 98K                      |
| Txns: 23  Voids: 0      Txns: 18  Voids: 2 (!)              |
| TILL 3 - Fatuma         TILL 4 - (empty)                    |
| CLOSED 10:00-14:00      NOT OPENED                          |
| ALERT: Till 2 has 2 voids today - review needed             |
+-------------------------------------------------------------+
```

16 Error Handling and Edge Case Specifications

16.1 Standard Response Envelope

```
// Error
{ "success": false,
  "error": { "code": "INSUFFICIENT_STOCK", "message": "Only 3 units of Paracetamol available",
             "field": "items[0].qty", "details": { } },
  "request_id": "req_abc123" }

// Success
{ "success": true, "data": { }, "meta": { "page": 1, "per_page": 50, "total": 284 } }
```

16.2 HTTP Status and Error Codes

HTTP	Error code	When it happens
400	VALIDATION_ERROR	Missing phone, negative price, malformed sync payload, etc.
400	INSUFFICIENT_STOCK	**Online sales only:** cart qty exceeds available batch qty. Never returned for offline-synced sales (see 16.3)
400	EXPIRED_STOCK	**Online sales only:** FEFO reached an expired batch. Offline sales are accepted and flagged
401	UNAUTHORIZED	Token not sent or malformed
401	INVALID_PIN	Login attempt with wrong PIN
403	PERMISSION_DENIED	Cashier trying to access reports
403	SUPERVISOR_REQUIRED	Void/refund override attempted without approval
404	NOT_FOUND	Product ID, sale ID not found
409	DUPLICATE_BARCODE	Adding product with existing barcode
409	TILL_ALREADY_OPEN	Double open attempt
422	MPESA_TIMEOUT	Customer did not respond in 60s
422	MPESA_REJECTED	Customer cancelled on phone
422	LOAN_ALREADY_ACTIVE	New application for borrower with an active loan
422	NHIF_CLAIM_DUPLICATE	Claim already submitted for this Rx
429	RATE_LIMITED	Brute force / sync loop
500	INTERNAL_ERROR	Unhandled exception
503	SYNC_QUEUE_FULL	Device offline for days with heavy use

Sync result statuses (not HTTP errors): `committed`, `committed_with_conflict`, `duplicate`, `quarantined`, and `rejected` (malformed payload only).

16.3 Critical Edge Cases and How They Are Handled

Scenario	Handling
Offline sale: item price changed while offline	Device caches selling prices at login. On sync the system compares cached vs current price. The sale is **always accepted at the offline price**. If the difference is over 5%, a PRICE_CHANGED conflict is logged and the owner gets a WhatsApp alert.
Two devices sell the last unit while offline (REVISED)	Both sales are committed because the goods have already left the shop. Stock goes negative, a NEGATIVE_STOCK row is written to `sync_conflicts`, and the owner is alerted. The next stock take resolves it. No sale is rejected or reversed.
Two online devices sell the last unit simultaneously	Online sales use `lockForUpdate()` row locks on `product_batches`. One transaction wins; the other gets INSUFFICIENT_STOCK and the UI shows 'Stock updated, only X units remain, please review cart.'
M-Pesa callback arrives twice	Every M-Pesa transaction stores `checkout_request_id`. A repeat callback returns 200 OK (to stop retries) but creates no second payment record.
M-Pesa paid but sale not completed	A reconciliation job runs every 15 minutes matching unmatched M-Pesa transactions to open sessions. Owner is alerted if any unmatched payment remains after 1 hour.

Scenario	Handling
Drug batch recall (Pharmacy)	All sale_items linked to the batch are identified and a patient contact list is generated. Stock is frozen online. If an offline device sells a recalled batch before syncing, the sale is committed, flagged RECALLED_BATCH, and added to the contact list.
Till variance at end of day	If actual cash differs from expected by more than TSh 5,000, the till cannot be closed without supervisor approval after reviewing each void/discount.
Loan repayment exceeds outstanding balance	Repayment is capped at the outstanding balance. The excess is flagged 'overpayment'; the owner decides to refund or credit it to the next loan.
Ingredient runs out mid-service (Restaurant)	Menu items using that ingredient become 'unavailable' on all waiter devices and the kitchen sees a red banner. Dishes already in active orders are unaffected.
Farmer credit: harvest not paid (Agro)	At harvest start all harvest-linked credit is flagged and field officers assigned. After 30 days unpaid the guarantor gets an SMS; after 60 days the account escalates to the owner.
Session token expires during active sale	POS uses a long-lived offline token (24h) separate from the API token (1h). The sale completes locally and syncs after the next refresh.
Offline operation from an unapproved device or an expired offline token (NEW)	The operation is **quarantined** (stored, not applied) and the owner is alerted to approve or discard it. It is never silently dropped.

17 Data Migration Strategy

Most clients migrate from handwritten books, Excel spreadsheets, or a legacy POS. Each needs a different approach. Migration is designed to complete in one evening with zero downtime.

Scenario	Source	Method	Time
Manual / handwritten	Paper stock books, notebooks	Manual entry via bulk import UI	2-4 hours
Excel / Google Sheets	.xlsx or .csv files	CSV import tool, auto-maps columns	30-60 minutes
Legacy POS	Exported CSV/Excel	CSV import + field mapper	1-2 hours
Completely new business	Nothing	No migration needed	15 minutes

17.1 CSV Import: Field Mapping

```
// Minimum required columns (products):
"name"            // REQUIRED
"selling_price"   // REQUIRED
// Optional: barcode, buying_price, category, current_stock, reorder_level, supplier_name, unit
// Pharmacy: expiry_date and batch_number REQUIRED; tfda_reg recommended

// Auto-mapping example:  Name -> name | Price -> selling_price | Stock -> current_stock | Category -> category
```

17.2 Migration Steps

Step	What happens
1. Export	Client exports product and customer lists from the old system or Excel; admin provides export guides for common systems
2. Upload	Upload CSV in Admin Panel (Client > Import Data); system shows column auto-mapping preview
3. Review mapping	Unmatched columns flagged yellow; system shows e.g. '284 products detected, 3 rows have errors'
4. Fix errors	Rows with missing required fields are highlighted and corrected inline; nothing is written until all errors are resolved
5. Dry run	Client previews every product name, price and stock level before committing
6. Commit	All records are written in a single database transaction: everything succeeds or nothing is written
7. Opening balances	Owner enters opening till float and supplier balances, and verifies stock counts against shelves
8. Go live	First real sale can be made; old system stays read-only for 30 days

17.3 What Cannot Be Migrated

Data type	Why	Recommended action
Transaction history	Different sale IDs, receipt formats, tax structure	Start fresh at go-live; keep old system for history
M-Pesa transaction history	Gateway provides current account transactions only	Export statement from Safaricom/Vodacom for records
NHIF claim history	NHIF has its own portal history	Keep old claims in the NHIF portal; new claims go through BiziSmart
Loan repayment history	Each loan has a unique schedule	Enter active balances as 'top-up existing loan' records
Employee attendance history	No standard format	New attendance tracking starts at go-live

18 Client Onboarding Flow

Onboarding is designed to get a new client operational in under 2 hours. The Super Admin handles setup; the client never configures servers or installs software.

18.1 Onboarding Steps: Admin Side

Step	Action
1. Client signs up / pays	Client pays by M-Pesa or bank; admin records payment and sets plan tier and start date
2. Create client account	Client Management > Add New Client: business name, vertical, owner phone, subdomain, plan tier
3. System auto-provisions (REVISED)	Creates the subdomain, covers it with the wildcard SSL certificate, inserts the business row, and applies the vertical's Pennant feature flags. **Tenant data lives in shared tables, isolated by** `business_id` **and the** `BelongsToBusiness` **global scope** (no separate schema or database per client).
4. Configure features	Toggle features by plan; templates make this a 2-minute step
5. Create owner account	Owner user with temporary PIN; WhatsApp message with subdomain URL and PIN
6. Add branches	Multi-branch clients only
7. Data import	Follow Section 17 if migrating
8. Handover	WhatsApp: 'Your BiziSmart is ready! Visit <subdomain>, login with your phone number and temporary PIN. Please change your PIN on first login.'

18.2 Client First-Login Setup Wizard

The wizard cannot be skipped until the mandatory steps are complete.

Step	Screen	Fields / actions
1 of 6	Change PIN	New PIN, confirm PIN (mandatory)
2 of 6	Business details	Business name, address, phone, TIN number, VRN number
3 of 6	Add staff	Name, phone, role, auto-generated PIN; can add several or skip
4 of 6	Import or add products	Import from Excel/CSV, add manually, or skip
5 of 6	Till setup	Receipt header and footer, default receipt method (Print / WhatsApp / Both), currency TSh
6 of 6	You're ready	Checklist summary, system URL, Go to Dashboard

18.3 Post-Onboarding: First Week Support

Day	Action
Day 1	WhatsApp check-in: first sale done? If not, schedule a 20-minute call
Day 3	2-minute voice note showing how to read the sales report and best sellers
Day 7	Automated WhatsApp weekly summary: total sales, top 3 products, customers served
Day 14	Show one underused feature (debt tracking, reorder alerts) via voice note
Day 30	Renewal reminder 7 days before renewal, with payment instructions

19 System Design

19.1 High-Level Architecture

BiziSmart is a React single-page PWA talking to a Laravel REST API, with one MySQL database and Redis for cache and queues. Everything runs on a single VPS in Phase 1. No microservices, no Kubernetes.

```
CLIENT DEVICES: Phone (Chrome/Safari) | Laptop | Tablet
  React 18 PWA: Redux Toolkit + RTK Query + MUI | Workbox service worker | IndexedDB (offline, Phase 3)
                 |  HTTPS 443  (Authorization: Bearer device token)
                 v
NGINX: SSL (wildcard), rate limiting, subdomain routing
  {business}.bizismart.co.tz/      -> React build (static files, SPA fallback to index.html)
  {business}.bizismart.co.tz/api/  -> PHP-FPM (Laravel API)       tenant = subdomain
  api.bizismart.co.tz/v1/...       -> PHP-FPM (payment webhooks only; tenant found from the payment record)
  admin.bizismart.co.tz            -> Laravel + Filament Super Admin, IP-whitelisted + 2FA
                 |
                 v
LARAVEL API:  middleware -> controllers -> Services (SaleService, StockService, ...) -> Eloquent
        |                                                  |                   |
        v                                                  v                   v
   MySQL 8                                          Redis 7             Horizon workers (queues)
   (business_id scope, InnoDB transactions)         cache, rate limits,  receipts, WhatsApp/SMS, TRA fiscal,
                                                     queues              reports, reconciliation
        |
        v
Cloudflare R2: receipts, Rx photos, ID documents, backups
External: AzamPay/Selcom | Meta WhatsApp | Beem SMS | TRA E-Fiscal
```

19.2 Request Lifecycle: Completing a Sale

Step	Layer	What happens
1	React app	Cashier taps Complete Sale; an RTK Query mutation sends `POST /api/v1/sales` with the Bearer token
2	NGINX	TLS, rate limit, route `/api` on the business subdomain to PHP-FPM
3	Middleware	`ResolveTenant` (subdomain to business, sets `CurrentBusiness`), Sanctum authentication, `feature` check (Pennant), permission gate (`can_sell`)
4	Validation	Form Request validates the payload
5	SaleService	`DB::transaction`: lock batch rows with `lockForUpdate()`, check stock, insert sale and items, decrement batches, update customer balance, write activity log. Roll back on any failure
6	After commit	Dispatch jobs with `afterCommit()`: receipt PDF, optional customer message, TRA post (if enabled)
7	Response	JSON via an API Resource; React updates the store and shows the receipt

```
// app/Services/SaleService.php  (online path)
public function complete(array $data, User $cashier): Sale
{
    return DB::transaction(function () use ($data, $cashier) {
        $sale = Sale::create([
            'branch_id' => $data['branch_id'], 'cashier_id' => $cashier->id,
            'payment_method' => $data['payment_method'], 'status' => 'completed',
        ]);
        foreach ($data['items'] as $line) {
            $batch = ProductBatch::whereKey($line['batch_id'])->lockForUpdate()->firstOrFail();
            if ($batch->qty_remaining < $line['qty']) {
                throw new InsufficientStock($batch, $line['qty']);   // online path only
            }
            $batch->decrement('qty_remaining', $line['qty']);
            $sale->items()->create([...$line, 'batch_id' => $batch->id]);
        }
        $sale->update(['total' => $sale->items()->sum(DB::raw('qty * unit_price - discount'))]);
        GenerateReceipt::dispatch($sale->id, CurrentBusiness::id())->afterCommit();
        return $sale;
    });
}
```

19.3 Complete Sale Data Flow

#	Step	Detail
1	Scan product	Camera scan fills the search box; the app calls `GET /products?barcode=` and gets the product and available batches; FEFO batch pre-selected. The tenant scope filters every query
2	Cart	Cart lives in the Redux store on the device; no API call per item; totals, VAT and change computed live
3	Payment	Cash, M-Pesa STK push (AzamPay via HTTP client) or split
4	M-Pesa confirm	Webhook `POST /mpesa/callback` matched by `checkout_request_id` (unique index = idempotent); UI polls status every 3s up to 60s
5	Commit	`SaleService::complete` runs in one transaction with row locks (19.2). **Online failure returns INSUFFICIENT_STOCK**
6	TRA E-Fiscal	If enabled for the client, a queued job posts to TRA with backoff retries and stores the fiscal receipt number
7	Receipt	Queued job renders the PDF, stores it on R2, saves the URL
8	Customer message	WhatsApp only if the customer opted in; otherwise SMS summary or printed receipt. Always async
9	Nightly job	Scheduler recalculates sales velocity and flags products likely to stock out within 7 days (post-MVP forecasting)

19.4 Offline-First Sync Design (accept and flag)

Offline selling is built in **Phase 3** inside the same React app (Dexie.js over IndexedDB plus a Workbox service worker). The rule is unchanged: **a sale that already happened at the counter is always recorded.** Problems become flagged conflicts for the owner, never lost sales.

```
DEVICE (React PWA):  IndexedDB (products, customers, sync_queue)  <->  Service Worker
   OFFLINE: sales queued locally, receipt shown/printed at once      ONLINE: sync fires automatically
SERVER:  POST /sync/push  (ordered operations)      GET /sync/pull?since=
   Each operation: operation_id (UUID), type, payload, created_at, device_id
   IDEMPOTENCY: unique index on sync_operations.operation_id; duplicates acknowledged and ignored
```

Situation at sync	Behaviour
Stock goes negative	Sale committed; stock may go below zero; NEGATIVE_STOCK conflict logged; owner alerted
Batch expired or recalled	Sale committed and flagged (EXPIRED_BATCH / RECALLED_BATCH); recall patient list updated
Price changed by more than 5%	Sale committed at the offline price; PRICE_CHANGED conflict logged; owner alerted
Duplicate operation_id	Acknowledged as success; nothing applied twice
Malformed payload	Rejected with VALIDATION_ERROR
Unapproved device or token outside the offline window	Quarantined for owner review; never discarded

```
// Sync path (inside DB::transaction, per operation)
$batch = ProductBatch::whereKey($op['batch_id'])->lockForUpdate()->first();
$batch->decrement('qty_remaining', $op['qty']);          // may go negative; no CHECK constraint
if ($batch->fresh()->qty_remaining < 0) {
    SyncConflict::create(['operation_id' => $op['operation_id'], 'device_id' => $deviceId,
        'type' => 'NEGATIVE_STOCK', 'product_id' => $batch->product_id, 'batch_id' => $batch->id,
        'details' => ['qty_remaining' => $batch->qty_remaining], 'status' => 'open']);
    $status = 'committed_with_conflict';
}
```

Example: two devices sell the last 2 units offline. Device A syncs first (stock 2 to 0, `committed`). Device B syncs next (stock 0 to -2, `committed_with_conflict`, NEGATIVE_STOCK logged) and the owner gets a WhatsApp alert to do a stock count. Both sales are recorded.

Step	What happens
1. Goes offline	Service worker detects loss of connectivity; reads come from IndexedDB; sales are queued with status pending_sync

Step	What happens
2. Keeps working	Product and customer lookup work locally; receipt shown or printed immediately; local stock deducted
3. Reconnects	Background sync (or the Sync button) calls POST /sync/push with operations in order
4. Server processes	Idempotency check, apply the rules above, commit; return per-operation status
5. Pull changes	GET /sync/pull?since= merges server changes into IndexedDB
6. Owner informed	Any committed_with_conflict or quarantined operation triggers a WhatsApp summary; the log is in the dashboard

19.5 Security Design

Layer	Controls
1. Network	HTTPS everywhere, HSTS, Cloudflare DNS/DDoS protection, Laravel RateLimiter on Redis (login and API)
2. Authentication	PIN login (4-6 digits, hashed); Sanctum Bearer device tokens with expiry and rotation, kept in memory and IndexedDB (needed for offline); strict Content-Security-Policy and no third-party scripts to limit XSS; TOTP 2FA mandatory for Owner and Super Admin; new device needs owner approval; auto-logout after 30 min
3. Authorisation	Every API route checks permissions (spatie roles, Gates/Policies); the React UI hides forbidden actions but the API is the authority; supervisor PIN for voids, refunds, large discounts; Super Admin on a separate hostname with IP whitelist
4. Tenant isolation	`BelongsToBusiness` global scope fails closed; tenant-aware jobs; tenant-prefixed cache keys and file paths; CI tests block any leak (19.5.1)
5. Data protection	Encrypted casts for national IDs, borrower GPS and prescription photo paths; R2 signed URLs expire after 1 hour; encrypted backups; activity log on every write; separate DB user without schema-change rights

19.5.1 Tenant Isolation Without RLS

Why this matters: MySQL has no row-level security, so isolation is enforced in application code. That is safe only if it is impossible to forget. The trait below makes tenancy automatic, **fails closed** when no tenant is set, and the tests make a leak a failed build.

```
// app/Models/Concerns/BelongsToBusiness.php
trait BelongsToBusiness
{
    protected static function bootBelongsToBusiness(): void
    {
        static::addGlobalScope('business', function (Builder $q) {
            $id = CurrentBusiness::id();
            $id ? $q->where($q->getModel()->getTable() . '.business_id', $id)
                : $q->whereRaw('1 = 0');                        // no tenant set => no rows
        });
        static::creating(function (Model $m) {
            abort_if(! CurrentBusiness::id(), 500, 'No tenant context');
            $m->business_id = CurrentBusiness::id();            // never trust client-supplied business_id
        });
    }
}

// app/Http/Middleware/ResolveTenant.php
public function handle($request, Closure $next)
{
    $sub = explode('.', $request->getHost())[0];
    $business = Business::withoutGlobalScopes()->where('subdomain', $sub)->where('is_active', true)->firstOrFail();
    CurrentBusiness::set($business->id);
    return $next($request);
}

// Queue jobs must carry the tenant:
class GenerateReceipt implements ShouldQueue {
    public function __construct(public int $saleId, public int $businessId) {}
    public function handle() { CurrentBusiness::using($this->businessId, fn () => /* work */ null); }
}
```

- Every tenant table has `business_id` and its model uses `BelongsToBusiness`.
- **Webhook routes** (e.g. `/mpesa/callback` on `api.bizismart.co.tz`) have no subdomain. They skip `ResolveTenant`, look up the payment by `checkout_request_id` with scopes removed, then set `CurrentBusiness`

before doing any work.

- `withoutGlobalScopes()` is allowed only in the Super Admin (Filament) namespace and `ResolveTenant`; a CI test greps for it elsewhere.
- Raw `DB::table()` and `DB::select()` on tenant tables are forbidden outside reviewed report classes, which must include `business_id` in the query.
- Cache keys include the business id (`products:{business_id}`); file paths start with `businesses/{business_id}/`.
- The application MySQL user cannot ALTER or DROP tables; migrations run with a separate user at deploy time.

```
// tests/Feature/TenantIsolationTest.php  (CI-blocking)
it('never returns another business rows', function () {
    $a = Business::factory()->create();  $b = Business::factory()->create();
    CurrentBusiness::using($a->id, fn () => Product::factory()->count(3)->create());
    CurrentBusiness::using($b->id, fn () => Product::factory()->count(2)->create());
    CurrentBusiness::using($a->id, fn () => expect(Product::count())->toBe(3));
    CurrentBusiness::using($b->id, fn () => expect(Product::count())->toBe(2));
    expect(Product::count())->toBe(0);                          // no tenant => fails closed
});

it('requires the trait on every model that has business_id', function () {
    foreach (tenantModels() as $class) {                        // models whose table has business_id
        expect(class_uses_recursive($class))->toContain(BelongsToBusiness::class);
    }
});
```

Upgrade path: if the client base or compliance needs grow, move to PostgreSQL and add row-level security as a second layer behind this scope. Eloquent code stays the same.

19.6 Notification and Communication Flow

WhatsApp (primary, owner-first) and SMS (fallback), sent through Laravel Notifications with custom `WhatsAppChannel` and `BeemSmsChannel`. Everything runs on Redis queues (`default`, `notifications`, `fiscal`, `reports`) supervised by Horizon, with retries and `backoff()`, and failed jobs kept in `failed_jobs` for review. Nothing is sent synchronously.

Trigger	Recipient	Channel	Timing	Content
Sale completed	Customer (opt-in)	WhatsApp, else SMS or print	Immediate	Receipt link + total
Debt reminder	Customer	WhatsApp	Day 1, 3, 7 overdue	Outstanding balance + payment instructions
Loan overdue	Borrower	WhatsApp + SMS	Day 1, 3, 7	Amount due, deadline, office contact
Guarantor alert	Guarantor	SMS	Day 7 overdue	Borrower name, amount, request to intervene
Low stock	Owner	WhatsApp	At reorder level	Product, qty, suggested order
Morning digest	Owner	WhatsApp	7:00 AM daily (scheduler)	Yesterday's sales, top products, pending actions
Drug batch recall	Owner	WhatsApp	Immediate	Drug, batch, affected patient count
Sync conflict	Owner	WhatsApp	Within 5 min	Conflict type, product, device; 'sale recorded, review stock'
Till variance	Owner	WhatsApp	End of day	Till, expected, actual, variance
Subscription expiring	Owner	WhatsApp	7 days before	Plan, expiry, payment instructions
TRA filing due	Owner	WhatsApp	3 days before	Filing period, estimated tax
New device login	Owner	WhatsApp	Immediate	Device, location, time; approve or block

19.7 Scalability Plan

Server names, specs and prices are targets; verify current Hetzner plans before quoting costs.

Phase	Clients	Setup	Monthly cost
1	0-20	Single VPS (target 4 vCPU, 8 GB RAM): NGINX + PHP-FPM + MySQL + Redis + Horizon	TSh 15,600
2	20-100	Bigger VPS (8 vCPU, 16 GB); MySQL read replica for report queries (Laravel read/write connections)	TSh 31,200
3	100-300	Server 1: NGINX + PHP-FPM + workers; Server 2: MySQL + Redis; Cloudflare CDN for static assets	TSh 62,400
4	300+	Load balancer, 2 app servers, MySQL primary + replicas, Redis, dedicated queue worker servers; consider Laravel Octane	TSh 125K+

19.7.1 Connections and Caching

- **Connections:** each PHP-FPM worker holds at most one DB connection per request. Keep `pm.max_children` (start around 20-30 on 8 GB) below MySQL `max_connections` with headroom for Horizon workers; tune with load testing.
- **Report load:** send heavy reports to the read replica or run them as queued jobs so they never slow the POS.
- **Cache:** product catalogue per business for 5 minutes (invalidate on update), permission lookups, rate-limit counters. **Never cache** sales or stock quantities.

```
// config/database.php  (Phase 2: read replica)
'mysql' => [ 'read' => ['host' => [env('DB_READ_HOST')]], 'write' => ['host' => [env('DB_HOST')]],
             'sticky' => true, /* ... */ ],

// Cache key always includes the tenant
Cache::remember('products:' . CurrentBusiness::id(), 300, fn () => Product::all());
```

19.8 Phase 1 Infrastructure and Deployment

```
VPS (Ubuntu 24 LTS, Hetzner; verify current plan)   4 vCPU / 8 GB / 80 GB SSD
  NGINX 80/443: / -> React build in /var/www/bizismart-spa (SPA fallback)
                /api and api.bizismart.co.tz -> PHP-FPM (Laravel public/); admin.bizismart.co.tz -> Filament
                SSL: Let's Encrypt wildcard (auto-renew)
  PHP-FPM + Laravel app      MySQL 8      Redis 7      Supervisor -> Horizon (queue workers)
  cron: * * * * * php artisan schedule:run     (nightly digests, reconciliation, backups)
  MONITORING: UptimeRobot (5-min ping, WhatsApp alert) + Laravel Pulse
  BACKUPS: spatie/laravel-backup daily 2 AM -> encrypted -> Cloudflare R2
  LOGS: storage/logs (daily rotation, 30-day retention)
External: Cloudflare R2 | AzamPay/Selcom | Meta WhatsApp | Beem SMS | TRA E-Fiscal

# nginx (per business subdomain, wildcard server_name)
server {
    server_name ~^(?<tenant>[a-z0-9-]+)\.bizismart\.co\.tz$;
    root /var/www/bizismart-spa;                        # React build
    location /api/ {
        root /var/www/bizismart/backend/public;
        try_files $uri /index.php?$query_string;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME /var/www/bizismart/backend/public/index.php;
        include fastcgi_params;
    }
    location / { try_files $uri /index.html; }          # SPA routing
    add_header Content-Security-Policy "default-src 'self'; img-src 'self' data: https:; connect-src 'self'";
}

# /etc/supervisor/conf.d/horizon.conf
[program:horizon]
command=php /var/www/bizismart/artisan horizon
user=www-data
autostart=true
autorestart=true
stopwaitsecs=3600

# deploy.sh  (run by GitHub Actions over SSH)
git pull origin main
(cd backend && composer install --no-dev --optimize-autoloader)
(cd frontend && npm ci && npm run build && rsync -a --delete dist/ /var/www/bizismart-spa/)
cd backend
php artisan migrate --force
php artisan config:cache && php artisan route:cache && php artisan view:cache && php artisan event:cache
php artisan horizon:terminate        # Supervisor restarts it with the new code
```

```
# .env (never commit real values)
APP_ENV=production   APP_DEBUG=false   APP_URL=https://bizismart.co.tz
DB_CONNECTION=mysql  DB_HOST=127.0.0.1  DB_DATABASE=bizismart  DB_USERNAME=biz_app  DB_PASSWORD=***
REDIS_HOST=127.0.0.1  QUEUE_CONNECTION=redis  CACHE_STORE=redis  SESSION_DRIVER=redis
SESSION_DOMAIN=admin.bizismart.co.tz  SESSION_SECURE_COOKIE=true   # sessions only for the Filament admin
AZAMPAY_CLIENT_ID=***  AZAMPAY_SECRET=***  AZAMPAY_CALLBACK_URL=https://api.bizismart.co.tz/v1/mpesa/callback
META_WA_TOKEN=***  META_WA_PHONE_ID=***  BEEM_API_KEY=***  BEEM_SECRET=***
FILESYSTEM_DISK=r2  R2_BUCKET=bizismart-files  R2_ACCESS_KEY=***  R2_SECRET_KEY=***  R2_ENDPOINT=***
```

19.9 System Design Summary

Component	Technology	Why this choice
Backend	Laravel (PHP) JSON API	One framework covers auth, ORM, queues, scheduler, testing; easy for one developer plus AI to maintain
Frontend	React 18 + Redux Toolkit + MUI + Workbox	Fast installable PWA that can work offline; all rules stay in Laravel
Super Admin	Filament	Client, plan and feature management built fast
Database	MySQL 8	Familiar, solid, transactions and row locks for stock accuracy
Multi-tenancy	`business_id` + global scope + CI leak tests	Fails closed; PostgreSQL RLS is the upgrade path
Queues	Laravel queues + Redis + Horizon	Async notifications, receipts, TRA posting; never blocks a sale
File storage	Cloudflare R2	Low cost, signed URLs
Offline	Dexie/IndexedDB + service worker (Phase 3)	Offline sales with accept-and-flag sync
Payments	AzamPay / Selcom	M-Pesa, Tigo Pesa, Airtel Money
Notifications	Meta WhatsApp + Beem SMS	WhatsApp owner-first, SMS fallback
Process management	NGINX + PHP-FPM + Supervisor	Standard, well-documented Laravel deployment
Monitoring	UptimeRobot + Pulse	Uptime alerts, slow queries, failed jobs
Testing	Pest + Vitest + Playwright	Tenant isolation, money paths and key user flows covered automatically
Scalability	Vertical first, then split DB and workers	Single VPS to ~20 clients, upgrade, split at 100-300+